

PRÁTICA 4: Blocos lógicos, ULA e Memória de dados

Data: 24/09/2026

I. INTRODUÇÃO

A Prática 4 tem como objetivo principal a compreensão e a implementação em linguagem de descrição de hardware (Verilog) dos blocos lógicos operacionais essenciais para o processamento de dados em uma arquitetura de computadores, com ênfase na Unidade Lógica e Aritmética (ULA) e no sistema de Memória de Dados.

A ULA representa os "músculos" do processador, sendo o bloco lógico encarregado de executar as operações aritméticas e lógicas básicas necessárias para a resolução de instruções. A execução dessas operações ocorre em sincronia e é comandada por sinais de controle que definem qual transformação será aplicada às entradas. Em conjunto, são explorados os conceitos de memória de dados, estruturas vitais que permitem o armazenamento e a recuperação de informações (dados e instruções) utilizando arrays de posições indexadas por endereços com restrições de alinhamento por bytes.

II. PROJETO DOS BLOCOS LÓGICOS E SIMULAÇÕES

Questão 1: Somador Parametrizável (16 bits)

O objetivo inicial do projeto consistiu na pesquisa das palavras reservadas `generate` e `parameter` em Verilog e na aplicação desses conceitos para a construção de um somador completo com dois operandos de 16 bits.

- O uso de `parameter` permite configurar dinamicamente o tamanho do barramento, tornando o design genérico e reutilizável, enquanto o `generate` possibilita instanciar múltiplos submódulos em um laço de repetição dinâmico durante a síntese de hardware.
- O módulo `parameterAdder.v` recebe o parâmetro de tamanho $N = 16$ e emprega um laço `for` dentro do bloco `generate` para instanciar 16 vezes o módulo de

somador completo de 1 bit (`somador.v`), conectando os lógicos de *carry in* e *carry out* em cascata.

- O módulo `somador.v` executa as funções booleanas fundamentais para o cálculo da soma (S) e do transporte (Cout) de operandos de 1 bit.
- O módulo top-level `adder16.v` faz a integração recebendo os operandos através das chaves da placa (SW) e direciona as saídas fracionadas para o módulo `decodificador.v`.
- O circuito `decodificador.v` traduz os sinais binários a fim de ativar corretamente os segmentos dos displays de 7 segmentos da placa FPGA Altera.

Análise do Circuito e Demonstração

Conforme diretriz do projeto, as principais decisões arquiteturais desta etapa priorizaram a síntese para a placa FPGA DE-10 Altera. O circuito teve seu funcionamento plenamente validado na prática em sala de aula por meio da visualização do resultado das somas em decimal nos displays de 7 segmentos da placa, confirmando o sucesso estrutural do bloco somador parametrizável.

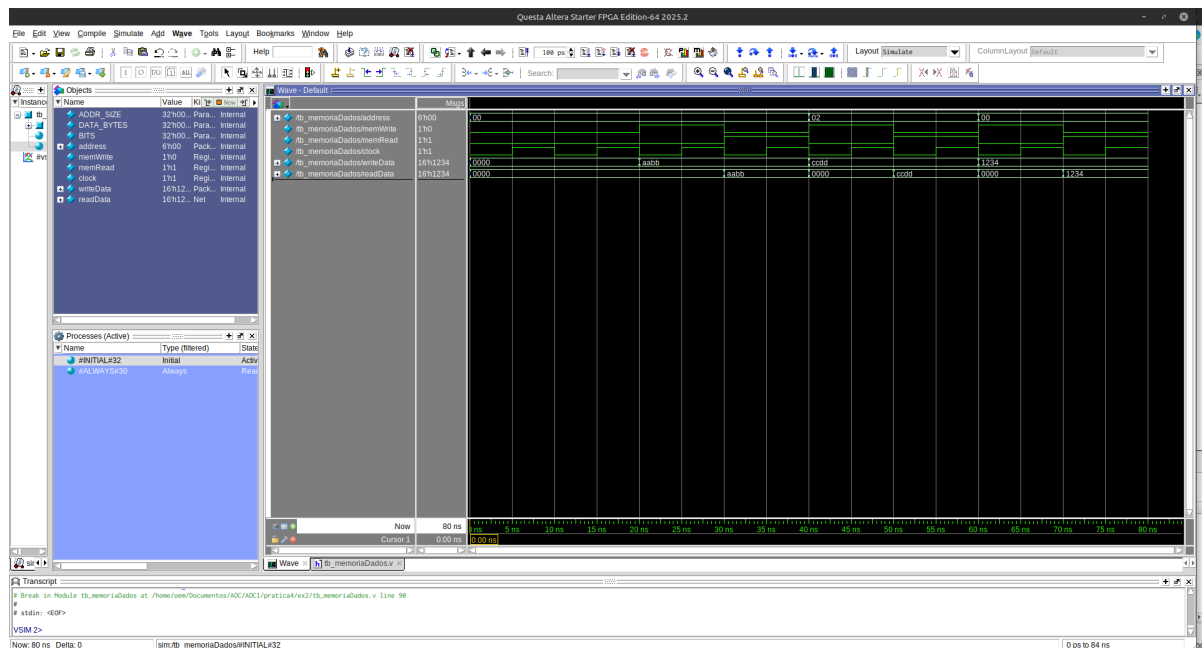
Questão 2: Memória de Dados

Foi desenvolvida uma memória de dados parametrizável baseada na restrição de alinhamento exigida pela arquitetura MIPS.

- O circuito projetado no arquivo `memoriaDados.v` possui uma capacidade de armazenamento fixada em 64 bytes por meio da limitação de endereçamento (`ADDR_SIZE = 6`).
- Devido à estrutura do MIPS, definiu-se a palavra de armazenamento com o tamanho de 16 bits (representando 2 bytes por palavra através do parâmetro `DATA_BYTES = 2`).
- A lógica de escrita implementada é estritamente síncrona; o armazenamento do dado de entrada na posição apontada pelo barramento acontece exclusivamente na borda de subida do clock e depende da ativação do sinal de habilitação `memWrite`.
- A lógica de leitura (`readData`) foi projetada de forma combinacional (assíncrona), operando livre de ciclos de clock sempre que o controle `memRead` se encontra em nível lógico alto, recuperando instantaneamente a concatenação dos bytes requisitados.

Análise da Simulação (Memória de Dados)

Na forma de onda gerada pelo testbench `tb_memoriaDados.v`, observa-se a coerência dos ciclos de leitura e escrita. Ao acionar a flag `memWrite`, o circuito demonstra a gravação dos valores hexadecimais de forma sincronizada com as bordas de subida do clock (por exemplo, a injeção da palavra `16'hAABB` no endereço 0 e o dado `16'hCCDD` no endereço 2). Imediatamente após a ativação exclusiva do comando `memRead`, os barramentos de saída expõem os valores armazenados de modo assíncrono ao se alterar o endereço para 0 e depois para 2, validando a mecânica de acesso aleatório do hardware. O final da simulação evidencia a corretude da operação de sobrescrita, onde o endereço 0 tem o antigo valor `AABB` atualizado permanentemente para o conteúdo `1234`.



Questão 3: Unidade Lógica e Aritmética (ULA)

Implementou-se a Unidade Lógica e Aritmética do processador MIPS utilizando um design parametrizável (`ULA.v`) estipulado para manipular informações de 32 bits ($N = 32$).

- A ULA foi capacitada a discernir e executar 5 instruções fundamentais baseando-se no código seletor de 3 bits `ALUOp`: as lógicas AND (000) e OR (001), a adição ADD (010), a subtração SUBTRACT (110) e a comparação SLT (111).
- Para economizar hardware, reaproveitou-se o módulo genérico `parameterAdder.v` tanto para operações de adição quanto de subtração e SLT. A operação de subtração é realizada utilizando o complemento de dois: o módulo inverte todos os bits do operando `b` e aciona o bit `Cin` do somador, comandado pelo bit mais significativo da entrada de controle (`ALUOp[2]`).

- O circuito detecta transbordamento (**overflow**) comparando discrepâncias entre os sinais dos operandos de entrada em contraste com o bit de sinal gerado pela operação.
- A funcionalidade Set on Less Than (SLT) é deduzida diretamente da subtração dos operandos, aferindo se o sinal remanescente acusa negatividade, o que indica que $a < b$, acoplando ainda uma verificação via XOR com o **overflow** para mitigar falsos-positivos provocados por estouro de capacidade.
- A flag indicadora de **zero** foi desenvolvida utilizando a porta de redução lógica ($\sim | \text{result}$), que varre o barramento indicando ativamente nível lógico alto unicamente se todas as vias convergirem a 0.

Análise da Simulação (ULA)

Com base na forma de onda simulada via Questa e gerada pelo **tb_ULA.v**, o circuito respondeu perfeitamente a todos os casos de teste estipulados:

1. **Soma (ADD = 010):** Ao somar 15 e 10, a ULA retornou o valor 25 com a flag zero desativada.
2. **Subtração (SUB = 110):** A operação de $25 - 10$ retornou 15 corretamente.
3. **Flag Zero:** Ao realizar a subtração de operandos idênticos ($42 - 42$), o resultado foi 0 e a flag zero foi ativada (1), validando o circuito comparador.
4. **SLT (111):** No teste em que $10 < 20$, o resultado retornado foi verdadeiro (1). Ao inverter a ordem ($20 < 10$), o resultado foi falso (0), comprovando o funcionamento da comparação.
5. **Overflow:** Ao somar 1 ao maior número positivo representável em 32 bits com sinal (2147483647), o valor estourou para a representação negativa (-2147483648), e a flag overflow foi ativada com sucesso (1).
6. **Operações Lógicas:** As instruções lógicas AND (000) e OR (001) aplicadas a máscaras hexadecimais corresponderam às interseções e uniões lógicas esperadas (ex: obtendo 16711680 e -1 na representação com sinal no simulador).

valor representasse -234 em um barramento de 32 bits (extensão em complemento de dois confirmada).

